

Week 1 Storage Benchmark Report

Three layouts are evaluated: A unpartitioned, B partitioned by local pickup_date, and C by pickup_year plus pickup_month. The saved measurements favor A for ingestion time, B for space and a single-day query, and C for the three required full-range queries. They do not establish a universal winner.

Evidence and methodology

Values below are retained from the pre-existing report, preserved as evidence/benchmark_report_original.md. It identifies run a6b890b6f2d64b18958ef485831f4187, schema 2.0.0, 9,394,330 accepted trips, Python 3.12.14, Spark 4.0.1 and Delta 4.0.1, and names week1_benchmark_metrics_1788724433878136100.json. That JSON, output tables and physical plans are absent from this checkout. These are historical reported results, not a fresh experiment or independently audited raw measurements. CPU, RAM, disk, Java version and historical effective writer count cannot be established from this report.

The report describes the method implemented in src/storage/benchmark.py: separately read raw taxi Parquet for each layout and time schema checks, casts, transforms, disk materialization, validation, deduplication, equal writer redistribution and Delta write. Shared Bronze, quarantine and metadata writes are excluded. Post-write count verification and directory inspection are outside the timer. Each layout uses a new run-ID directory.

Storage results

Metric	A Unpartitioned	B Daily	C Monthly
Taxi ingestion seconds	217.941	225.304	228.606
Directory size MiB	1039.31	921.19	1024.12
Parquet data files	4	91	4

Directory size includes Delta metadata and other files; MiB means 1024 squared bytes. File counts above cover Parquet only. The current code also records total generated files and data-only bytes, but those values are not available in the surviving report. They must not be inferred from the Parquet counts.

Query results

Five timed repetitions follow one warmup per query/layout. Cells show mean +/- sample standard deviation in milliseconds. Spark cache is cleared before timed runs; the operating-system cache remains warm. Strategy order rotates across query types.

Query	A Unpartitioned	B Daily	C Monthly
Q1 Trips per pickup borough	2061.83 +/- 250.70	2199.73 +/- 119.54	1706.02 +/- 96.58
Q2 Average duration per local day	1163.60 +/- 44.10	1259.72 +/- 62.37	1094.35 +/- 123.13
Q3 Average fare per pickup borough	2007.37 +/- 145.60	2356.97 +/- 85.29	1918.50 +/- 165.21
Q4 February 14 borough analysis	1682.27 +/- 150.84	1177.44 +/- 92.30	1218.24 +/- 72.31
Q5 February borough analysis	1853.22 +/- 203.58	1559.21 +/- 135.07	1645.35 +/- 121.56

Q1-Q3 satisfy the assignment. Q1 and Q3 left-join zones and retain unmatched IDs under Unknown; Q2 averages rounded trip_duration_minutes. Q4 filters pickup_date; Q5 filters pickup_year/month. Identical SQL is run across layouts. The saved report records cross-layout equivalence as true; current code compares warmup results with floating-point tolerances.

Performance discussion

Unpartitioned ingestion is fastest in the saved experiment. Daily and monthly take 3.4% and 4.9% longer respectively. There is only one ingestion trial per layout and order is fixed, so these differences cannot be separated from cache and order effects.

Daily occupies 11.4% less directory space than unpartitioned and produces 91 Parquet files rather than four. Monthly saves only 1.5% relative to unpartitioned. More files are not automatically worse: partition-local organization can improve selective reads, while extra file-open and scheduling work can hurt broad scans. The figures alone do not isolate compression, ordering or metadata contributions.

Monthly has the lowest observed means on Q1-Q3, improving over unpartitioned by 17.3%, 6.0% and 4.4%. The smaller differences should be read alongside the standard deviations; they are not evidence of statistical significance. Daily has the lowest means on Q4 and Q5, 30.0% and 15.9% below unpartitioned. These warm-system results suggest testing daily partitions for narrow date access and monthly for the current broad analytical workload.

Do not infer partition pruning from query names or timings. Q4 filters `pickup_date`, whereas monthly physical partitions use `pickup_year` and `pickup_month`; Q5 uses the latter columns. Pruning and data skipping require plan/scan evidence. The missing physical plans prevent that audit, and scanned bytes are not measured by the current harness. Four monthly files are not evidence of four months or a particular number of writer tasks.

Decision and limits

Keep the configured monthly layout as a provisional choice for the Q1 platform and required full-range analytics. Reconsider it if single-day reads dominate. This is a local workload decision: warm repetitions are not independent cold-cache trials, no twentyfold experiment is available, and the saved results lack a complete hardware/configuration record. Do not combine older integrated-input timings with the raw-input method or describe layout ingestion as full platform ingestion.

Reproduction and stronger follow up

From the configured project environment run `python scripts/run_week1.py`. The runner generates fresh strategy directories, latest and timestamped JSON metrics, and `reports/week1_benchmark_report.md` on successful completion. Use `scripts/render_benchmark_report.py --metrics <successful-json> --output <report.md>` to regenerate a table report. It rejects failed, ingestion-only and historical integrated-input artifacts. See the root README for setup and full commands.

Current YAML uses `local[*]`, 8g requested driver memory, eight shuffle/writer partitions and five query repetitions; AQE is enabled by the session factory. These are current defaults, not verified settings of the saved experiment. File counts and timings can differ when rerun. Preserve the generated JSON, plans, configuration and input identities with the submission.

For stronger evidence, record hardware, disk, Java and input hashes; repeat ingestion with randomized order; add cold-cache trials where practical; report scanned bytes and plans; and compare twentyfold growth as both denser days and longer history. Reassess small files, writer parallelism and context broadcasts before drawing a scalability conclusion.

Sources: `reports/week1/evidence/benchmark_report_original.md`; `src/storage/benchmark.py`; `scripts/render_benchmark_report.py`; `scripts/run_week1.py`; `config/platform_config.yaml`. Percentages are calculated from the displayed historical values, rounded to one decimal place.